FIL GRADUATE AI COHORT · RAG LAB COMPANION

How the RAG Lab works, from question to answer

A beginner-friendly tour of the whole system: what each part does, why it exists, and how a question travels through it. With diagrams for every layer.

Basic RAG | MQR | MQR + RRF | Agentic RAG | Vectorless RAG

Covers Lab 1 (Basic RAG → MQR → RRF), Lab 2 (Agentic RAG with LangGraph) and Lab 3 (Vectorless RAG)

Stack: React · FastAPI · LangChain · LangGraph · FAISS · rank-bm25 · OpenAI SDK via the FIL AI Gateway

Models: azure/gpt-6-luna (chat) · azure/text-embedding-ada-002 (embeddings)

October 2026

START HERE

Contents

1	**The big picture** What the app does, the six layers, the RAG loop	all labs
2	**Seven ideas you need first** LLM, prompt, chunk, embedding, keyword search, ranking, citation	basics
3	**The knowledge base and its three indexes** Chunks + FAISS, the tree, the entity graph	Labs 1–3
4	**Talking to the AI models** llm.py, llm_client.py, the FIL AI Gateway, errors and limits	all labs
5	**Four ways to find evidence** Basic, MQR, MQR + RRF, Vectorless, side by side	Labs 1, 3
6	**Writing the answer** Grounded prompt, context budget, citations	Lab 1
7	**The agent** When a fixed pipeline is not enough: LangGraph nodes, tools, memory	Lab 2
8	**The backend: one request, start to finish** FastAPI endpoints and a request traced step by step	all labs
9	**The frontend: what each tab shows** Six tabs and what to look at in each	all labs
10	**Measuring: evaluation and comparison** Hit rate, MRR, cost per mode	Labs 1, 3
11	**Configuration, tests and project map** .env knobs, offline tests, trainer and student zips, folders	all labs
12	**Putting it together** One question through every mode, and which mode to use when	summary
A	**Glossary** Every term in this guide, in plain words	

START HERE

How to read this guide

The parts follow the order in which the system makes sense: first the big picture, then the data, then the model access, then the ways of finding evidence, then the answer, then the agent, and finally the API, the screens and how we measure. Each part opens with a **one-sentence summary**, has at least one diagram, and ends with the **files** to open and something to **try** in the running app.

Every diagram uses the same colours for the same layer of the system:

- **Browser** · React screens
- **Services** · run a question, write the answer
- **Agent** · LangGraph decisions
- **Data** · files and indexes
- **HTTP API** · FastAPI endpoints
- **Retrieval** · find the evidence
- **Models** · gateway, gpt-6-luna, ada-002

THE PICTURE TO KEEP IN MIND

Think of the app as a **helpdesk with a very fast reader**. The handbook is the library. Retrieval is the librarian who fetches the right pages. The LLM is the writer who answers *only* from the pages it was handed and says which page each fact came from. Each lab gives the librarian a better way to find pages.

PART 1

The big picture

The RAG Lab is a small but complete question-answering product over a company handbook: a React website, a Python API, several retrieval strategies, an agent, and an evaluation bench to compare them.

IN ONE SENTENCE

A user asks a question about the NovaPay employee handbook; the app **finds the relevant passages**, gives them to **azure/gpt-6-luna**, and returns an answer with **citations**, while showing exactly how the passages were found.

Why not just ask the LLM?

It has never seen the handbook

NovaPay is fictional and internal. The model cannot know its leave rules or error codes.

It would guess

Asked anyway, an LLM writes something plausible. In a helpdesk, a confident wrong answer is worse than none.

Checking must be possible

With citations, every fact points to a passage a person can open and verify.

RAG (Retrieval-Augmented Generation) solves all three: *retrieve* the right passages, *augment* the prompt with them, and let the model *generate* an answer from them only.

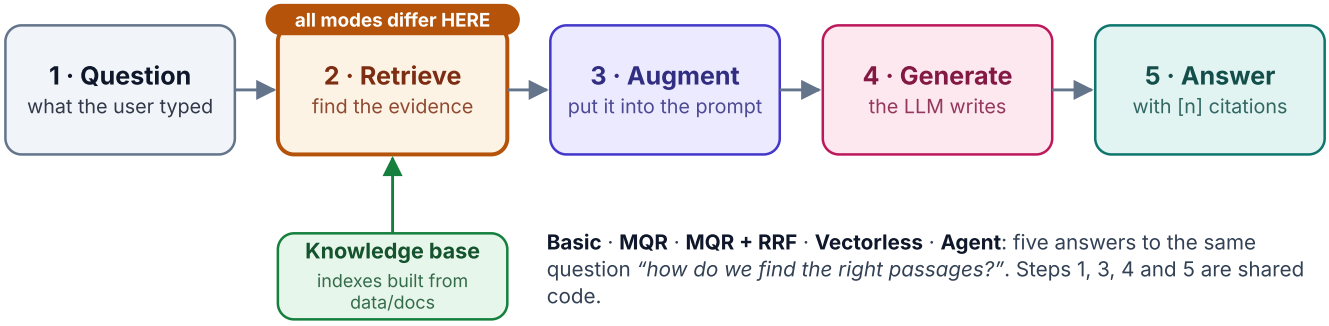


Figure 2 · The RAG loop. Every mode in this app runs these five steps; they differ only in how step 2 finds evidence.

The whole system on one page

Figure 1 zooms out from the loop to every part of the app. The six coloured layers are the same in every diagram that follows.

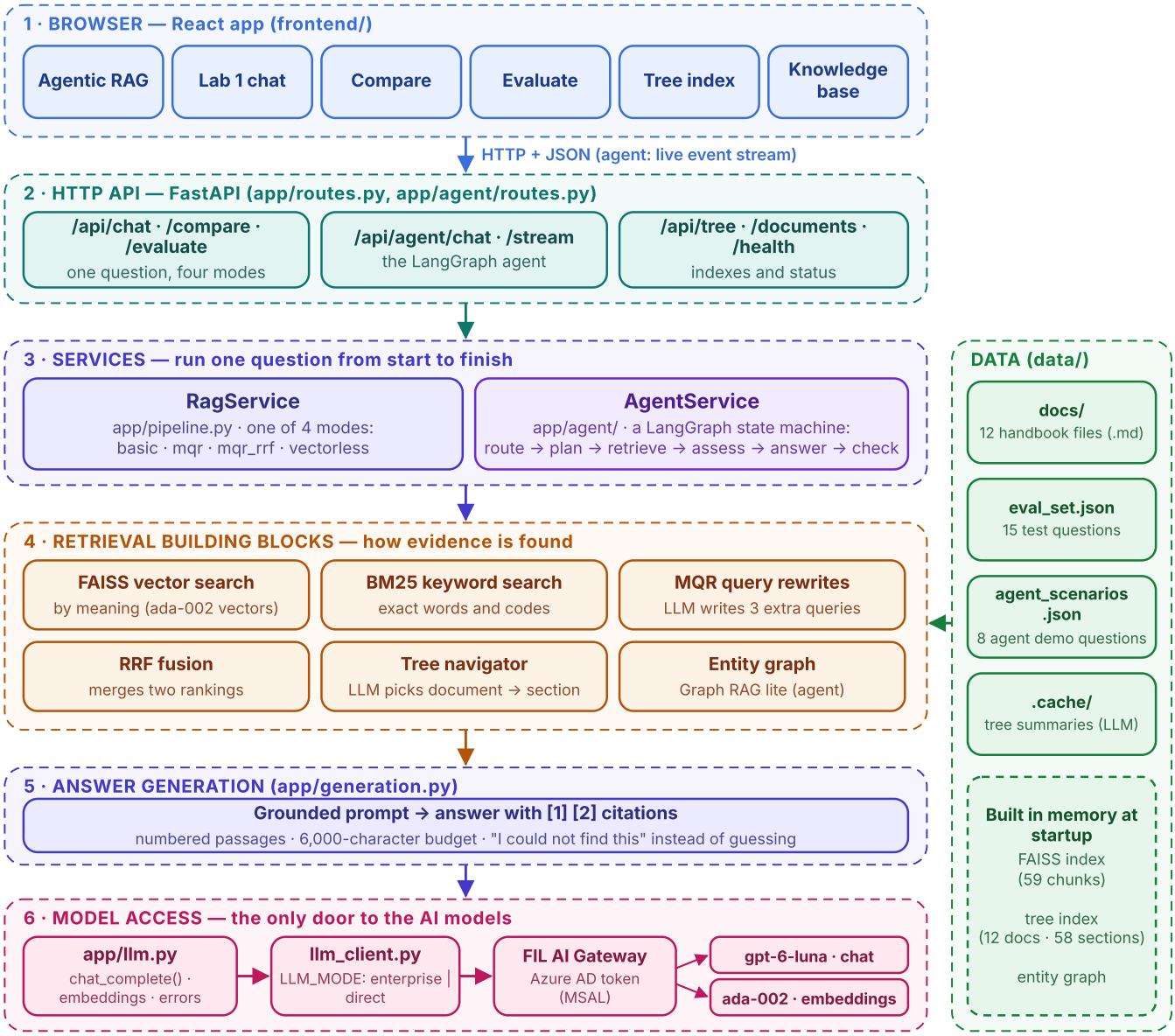


Figure 1 · The whole system on one page. Read it top to bottom: a question travels down through the layers and the answer travels back up.

The six layers in plain words

Layer	What it does	Where in the code
1 · Browser	Six tabs: ask questions, compare modes, run evaluations, browse the indexes.	`frontend/src/*.jsx`
2 · HTTP API	Receives JSON requests, checks them, calls a service, returns JSON (or a live event stream for the agent).	`app/routes.py`, `app/agent/routes.py`
3 · Services	**RagService** runs one of four fixed pipelines. **AgentService** runs the LangGraph agent.	`app/pipeline.py`, `app/agent/service.py`
4 · Retrieval	Reusable building blocks that find evidence: vector search, keyword search, query rewrites, fusion, tree navigation, entity graph.	`app/vector_store.py`, `mqr.py`, `rerank.py`, `vectorless/`, `agent/graph_rag.py`
5 · Generation	Turns passages into a numbered context and a strict prompt, then finds the citations in the reply.	`app/generation.py`

Layer	What it does	Where in the code
▢ 6 · Model access	The only code that talks to the models, inside FIL (gateway) or outside (API key).	`app/llm.py`, `llm_client.py`

FILES TO OPEN

`README.md` (what each lab adds) · `app/main.py` (starts everything) · `app/pipeline.py` (the four modes)

TRY IT

Start the app with `uvicorn app.main:app --port 8000` and open `http://127.0.0.1:8000`. The chips in the top right show the chat model, the embedding model, the FAISS chunk count and the tree size: that is layer 6 and the data column of Figure 1, live.

How the labs built this, layer by layer

Lab	What it added	Problem it solved
Lab 1	Basic RAG, then MQR (query rewrites), then RRF (fusion of meaning and keywords), plus grounded answers with citations	Wrong words in the question, exact codes, too much noise sent to the LLM
Lab 2	The agent: routing, planning, parallel retrieval, self-assessment, calculator, self-check, memory, clarifying questions	A fixed one-shot pipeline cannot decide, loop, calculate or ask back
Lab 3	Vectorless RAG: a tree index of the handbook that the LLM navigates; cost columns in Evaluate	“Similar” is not always “relevant”; structure is lost when text is chunked

PART 2

Seven ideas you need first

Everything in the app is built from a handful of ideas. If these are clear, every diagram in the rest of the guide reads easily.

IN ONE SENTENCE

An **LLM** writes text from a **prompt**; we cut documents into **chunks**, find the right ones by **meaning** (embeddings) or by **words** (BM25), **rank** them, and ask for **citations**.

1 · LLM (large language model)

A model that continues text. Here: **azure/gpt-6-luna**. It is good at reading and writing, but only knows what was in its training data and what we put in the prompt.

2 · Prompt

The text we send to the LLM: instructions (the *system* message) plus the question and the passages (the *user* message). The app's prompts live in `generation.py`, `mqr.py`, `agent/prompts.py` and `vectorless/prompts.py`.

3 · Chunk

A small piece of a document, here at most 500 characters, starting with its heading ("Leave Policy > Sick leave"). Small pieces fit in a prompt and can be searched one by one.

4 · Embedding (vector)

ada-002 turns a text into 1,536 numbers. Texts with similar meaning get similar numbers. **Cosine similarity** measures how close two vectors are (1.0 = same direction). **FAISS** finds the closest chunks fast.

5 · Keyword search (BM25)

A classic search formula: a chunk scores high when it contains the question's rare words. It does not understand meaning, but it never confuses `PAY-E4035` with `PAY-E4033`.

6 · Ranking and fusion

Every search returns a ranked list. **RRF** (Reciprocal Rank Fusion) merges two lists by position: `1 / (60 + rank)` from each list, added up. Being near the top of both lists wins.

7 · Grounding and citations

The answer prompt says: use only these numbered passages, write `[n]` after each fact, and say "I could not find this in the handbook" when the passages do not contain the answer.

Bonus · Two new ideas in Labs 2 and 3

Agent: an LLM that decides the next step (search again? calculate? ask the user?) inside a graph of steps. **Tree index:** the handbook's table of contents, which an LLM can navigate like a reader.

MEANING VERSUS WORDS

Embeddings are like asking a colleague "which pages talk about working from home?": they understand that "WFH" and "hybrid working" are the same topic. BM25 is like pressing Ctrl+F: exact, literal, and very good at codes. The best retrievers in this app use **both**.

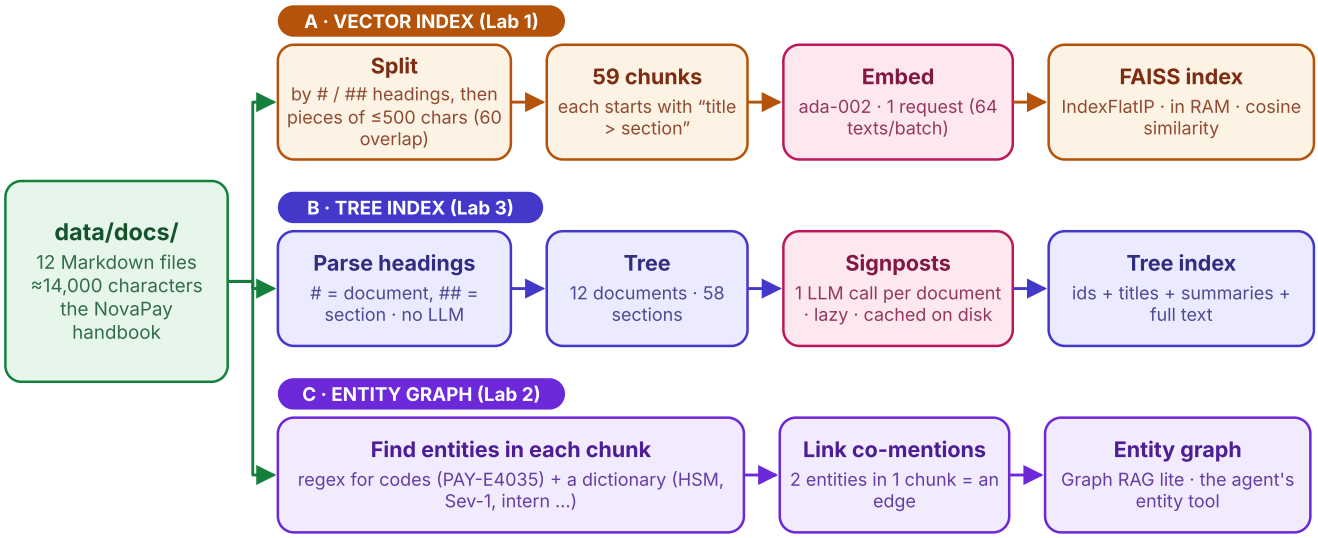
PART 3

The knowledge base and its three indexes

The handbook is 12 Markdown files in `data/docs/`. At startup the app reads them once and builds three different indexes, one for each way of searching.

IN ONE SENTENCE

The same 12 files become **59 chunks in FAISS** (search by meaning), a **tree of 12 documents and 58 sections** (search by navigation) and an **entity graph** (search by "everything about X").



BM25 has no stored index: it scores the chunks at question time, so it is always up to date and costs no model call.

Figure 3 · One folder, three indexes. All three are rebuilt when you upload a file or press Re-index.

A · The vector index (Lab 1)

`app/ingest.py` splits each file in two steps: first by its `#` and `##` headings, then into pieces of at most 500 characters with 60 characters of overlap. Every chunk starts with "document title > section" so both the embedding and BM25 know what it is about, and gets an id such as `04_payment_error_codes.md#2`.

`app/vector_store.py` sends the chunk texts to ada-002 (64 per request), normalises the vectors and stores them in a FAISS `IndexFlatIP`. With normalised vectors the inner product equals the cosine similarity. It also caches query vectors, so asking for the same query twice costs no second embedding call.

B · The tree index (Lab 3)

`app/vectorless/tree.py` reads the headings instead of cutting the text: each file becomes a document node, each `##` heading becomes a section node, and text before the first `##` becomes an "Overview" section. A file without sections (the glossary) is a single leaf. Building the tree needs no model at all.

```
root  NovaPay employee handbook
 ├─ [01] NovaPay Leave Policy (HR-POL-01)
 │    ├─ [01.0] Overview
 │    ├─ [01.1] Earned leave          … [01.5] Applying for leave
 ├─ [04] Payments Platform Error Codes (ENG-RUN-PAY)
 │    ├─ [04.1] PAY-E4031 - Settlement batch rejected after cut-off
```

```
│    ├─ [04.4] PAY-E4035 - Signature mismatch on bank file
└─ [11] NovaPay Engineering Glossary            (a leaf: no sections)
```

Each node then gets a **signpost**: a one-sentence summary that the navigator reads later. With `VECTORLESS_SUMMARIES=llm` the LLM writes all signposts of a document in one call (12 calls in total), only when vectorless is first used, and caches them in `data/.cache/tree_summaries.json` by document content. `extractive` uses first sentences (free); `none` shows titles only.

C · The entity graph (Lab 2)

`app/agent/graph_rag.py` finds named things in every chunk: error codes by pattern (`PAY-E4035`, `AUTH-E1107`), product names such as TripNova, and a dictionary of terms (HSM, Sev-1, graduate trainee, change freeze …). Two entities in the same chunk are linked. The agent's **entity tool** then returns every chunk that mentions an entity, one per document first, which top-5 similarity search cannot do for questions like “every rule for graduate trainees”.

FILES TO OPEN

`app/ingest.py` · `app/vector_store.py` · `app/vectorless/tree.py` · `app/agent/graph_rag.py` · `app/pipeline.py` (`reindex()`)

TRY IT

Knowledge base tab: open `04_payment_error_codes.md` to see its chunks. **Tree index** tab: expand document 04 to see the same text as sections with signposts. Upload a new `.md` file and watch both indexes rebuild.

PART 4

Talking to the AI models

Only two files in the whole project know how to reach a model. That keeps secrets in one place, makes the app work inside and outside FIL, and lets the tests swap the real models for fakes.

IN ONE SENTENCE

All code calls `chat_complete()` or `GatewayEmbeddings` in `app/llm.py`, which gets its client from `llm_client.get_client()`; one line in `.env` decides whether that client goes through the **FIL AI Gateway** or a normal API key.

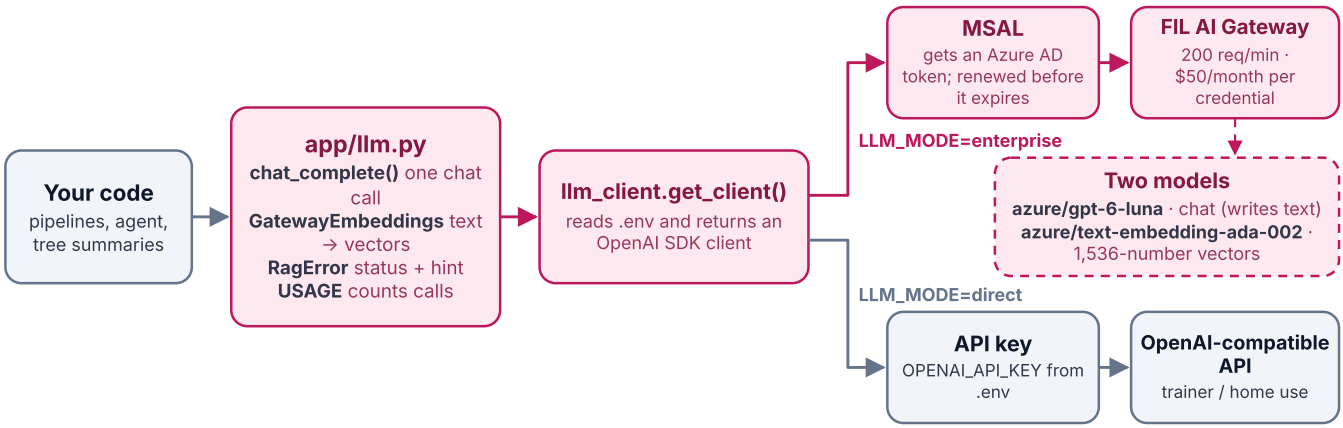


Figure 4 · Every model call goes through two files. Nothing else in the app knows where the models live.

The two modes

	LLM_MODE=enterprise (students, inside FIL)	LLM_MODE=direct (trainer, home)
Where calls go	`FIL_BASE_URL`, the FIL AI Gateway	`OPENAI_BASE_URL` or api.openai.com
How it signs in	MSAL gets an Azure AD token with the client id and secret; a fresh token is put on every request	`OPENAI_API_KEY`
Models	`azure/gpt-6-luna`, `azure/text-embedding-ada-002`	any chat + embedding model
Limits	200 requests/minute and $50/month per credential	your own account

When something goes wrong

`app/llm.py` turns every failure into a **RagError** with an HTTP status and a hint that the UI shows to the user:

What happened	Status	Hint shown
Model call timed out	504	Raise `LLM_TIMEOUT_SECONDS` in .env
Gateway not reachable	502	Check VPN, `FIL_BASE_URL` and `CA_BUNDLE`
Too many requests or budget used	502 (429 from the gateway)	Wait a minute and retry
Token rejected	502 (401/403)	Check the FIL client id, secret and `FIL_ENV`

What happened	Status	Hint shown
Model name unknown	502 (404)	Check the model names in .env
A lab TODO is not implemented yet	501	Which TODOs to implement, and run pytest

Two small counters help the comparison later: `GatewayEmbeddings.calls` counts embedding requests and `USAGE["chat"]` counts chat calls. Every answer and every evaluation reports both.

FILES TO OPEN

`llm_client.py` · `app/llm.py` · `.env.example` · `00_check_setup.py`

TRY IT

Run `python 00_check_setup.py`. It tests the token step and one chat call separately, so it tells you exactly which step is broken.

PART 5

Four ways to find evidence

This is the heart of the app. All four modes take the same question and produce a list of passages for the LLM; they differ in how they search, how much they send, and what it costs.

IN ONE SENTENCE

Basic takes the 5 nearest chunks; **MQR** searches with 4 phrasings; **MQR + RRF** keeps only the 5 best by meaning *and* keywords; **Vectorless** lets the LLM open the right document and section of a table of contents.

Lab 1 Mode 1 · Basic RAG

The question is turned into a vector and FAISS returns the 5 closest chunks. They are pasted into a deliberately naive prompt with no numbering and no citation rule. It is cheap and fast, and it is the "before" in every before/after comparison.

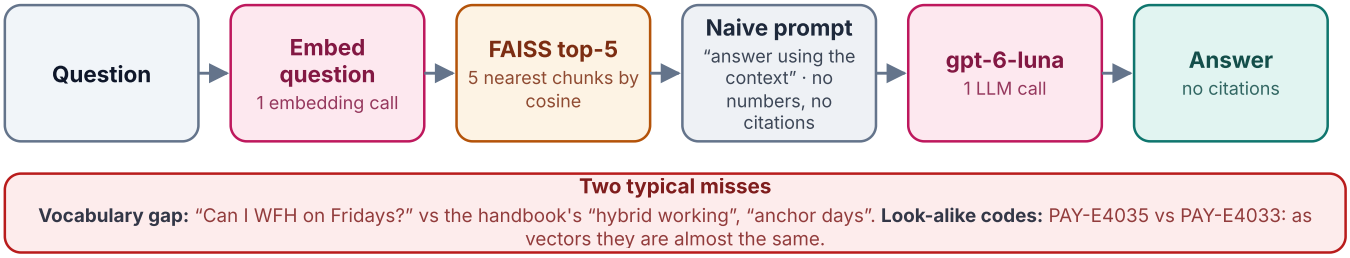
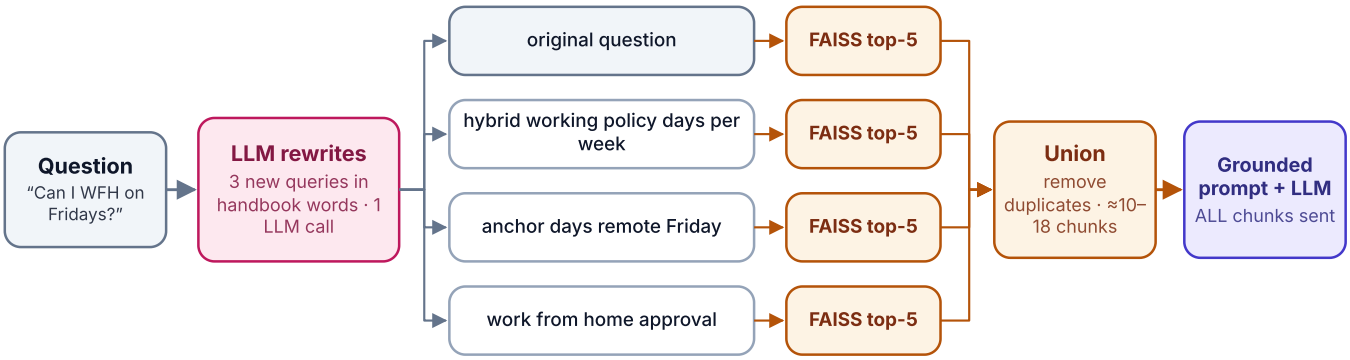


Figure 5 · Basic RAG (app/basic_rag.py). Cheapest and fastest; the baseline every other mode is compared with.

Lab 1 Mode 2 · Multi-Query Retriever (MQR)

The user's words are often not the handbook's words. MQR asks the LLM for 3 rewrites in formal handbook language (keeping codes such as `PAY-E4031` unchanged), searches FAISS with all 4 queries and takes the union. The rewrites are cached per question, so the Compare tab does not pay for them twice.



Fixes the vocabulary gap. **Costs** 1 extra LLM call, 4 embedding calls, and every chunk found goes to the LLM (more tokens, more noise).

Figure 6 · Multi-Query Retriever (app/mqr.py). More queries = more nets in the water = higher recall.

Lab 1 Mode 3 · MQR + Reciprocal Rank Fusion

MQR finds more but sends too much. RRF ranks the MQR pool twice, by meaning (the best cosine to any of the 4 queries, using cached vectors so no new embedding calls) and by keywords (BM25 over the pool), then

fuses the two rankings and keeps the top 5 within a 6,000-character budget. This is the default mode of the chat tab.

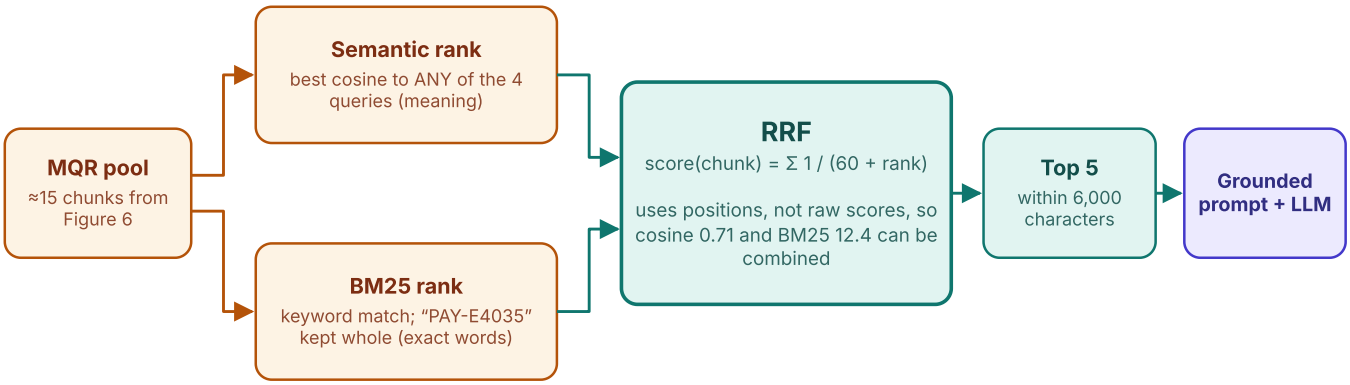


Figure 7 · MQR + Reciprocal Rank Fusion (app/rerank.py). Keep MQR's recall, then send only the best five.

Lab 3 Mode 4 · Vectorless RAG

Instead of asking "which chunks *sound* like the question?", vectorless asks "*where* in this handbook would the answer live?". The LLM sees the 12 documents (id, title, signpost) and picks up to 3; then it sees their sections and picks up to 4. The picked sections are read in full and go to the same grounded prompt as MQR + RRF. A step limit (`VECTORLESS_MAX_STEPS=3`) keeps cost bounded, and if a reply is unusable, BM25 picks the sections instead.

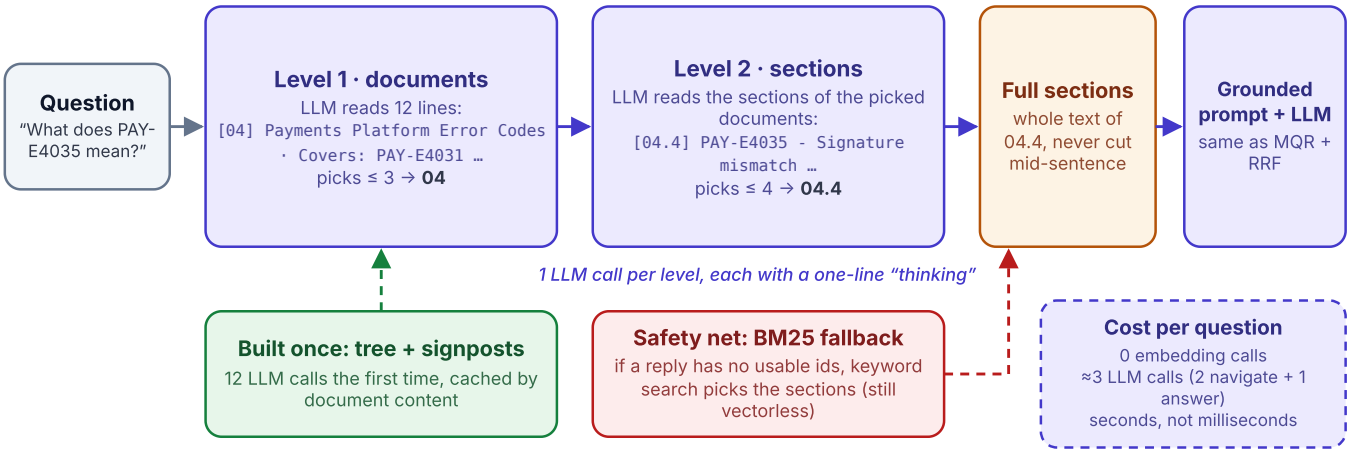


Figure 8 · Vectorless RAG (app/vectorless/). The LLM reads a table of contents and decides where to look. No embeddings, no FAISS.

The four modes side by side

	Basic	MQR	MQR + RRF	Vectorless
Finds evidence by	meaning (1 query)	meaning (4 queries)	meaning + keywords, fused	LLM reasoning over the outline
Unit sent to the LLM	5 chunks	all ≈10–18 chunks	top 5 chunks	up to 4 whole sections
LLM calls per question	1	2	2	≈3 (2 navigate + 1 answer)
Embedding calls per question	1	4	4 (shared with MQR when cached)	0

	Basic	MQR	MQR + RRF	Vectorless
One-off index cost	embed 59 chunks	same	same	12 LLM calls for signposts (cached)
Answer prompt	naive, no citations	grounded, citations	grounded, citations	grounded, citations
Typical strength	speed, cost	vocabulary gap	exact codes + vocabulary, small context	structure, multi-part questions, explainable path
Typical weakness	wrong words, look-alike codes	noise, tokens	still top-5 lookalikes	latency; a wrong turn at level 1

FILES TO OPEN

`app/basic_rag.py` · `app/mqr.py` · `app/rerank.py` · `app/vectorless/navigator.py` · `app/pipeline.py` (`run()`, `retrieve()`)

TRY IT

Lab 1 chat tab: ask “Can I WFH on Fridays?” in each mode and watch the **Retrieval inspector**: the queries searched, every chunk found, its semantic, BM25 and RRF rank, and which chunks reached the LLM. In Vectorless mode the inspector shows the navigation path instead.

PART 6

Writing the answer: grounded generation

Good retrieval is half the job. The other half is a prompt that keeps the model honest and makes every fact checkable.

IN ONE SENTENCE

The passages are numbered `[1] [2] …` in rank order inside a 6,000-character budget, the prompt demands a citation after every fact, and the app reads the citations back so the UI can highlight the passages that were used.

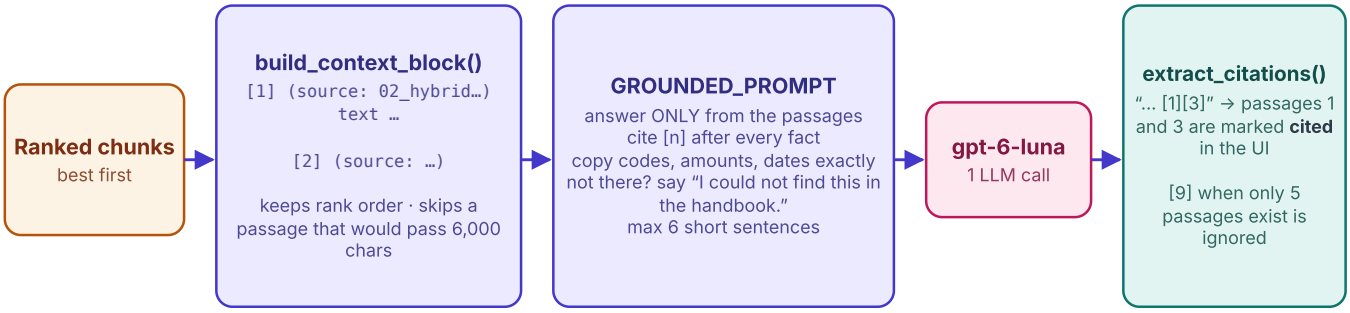


Figure 9 · Grounded generation (app/generation.py), shared by MQR, MQR + RRF, Vectorless and (in its own form) the agent.

The rules, and why each exists

Rule in the prompt	Why
Answer ONLY from the numbered passages	Stops the model from mixing in general knowledge that may be wrong for NovaPay.
Cite [n] after every fact	Every claim can be checked by opening one passage; uncited claims stand out.
Copy codes, amounts, dates and names exactly	"PAY-E4035" and "INR 25,000" must not be paraphrased.
Otherwise reply exactly "I could not find this in the handbook."	An honest "not found" is better than a confident guess.
Partly answered? Answer that part and say what is missing	Multi-part questions often have one part the handbook does not cover.

The budget. `build_context_block()` walks the ranked passages in order and skips any passage that would push the context over `MAX_CONTEXT_CHARS` (6,000), so the best passages always come first and the cost has a ceiling. MQR alone ignores the budget on purpose: sending everything it found is exactly the problem RRF fixes.

The citations. `extract_citations()` accepts `[1]`, `[1][3]` and `[1, 2]`, ignores numbers that do not exist (the model wrote `[9]` but there were 5 passages) and returns a clean sorted list.

FILES TO OPEN

`app/generation.py` · `frontend/src/ui.jsx` (citation chips you can click)

TRY IT

Ask something the handbook does not cover ("What is the canteen menu?") in MQR + RRF mode. Then click a citation chip in any answer: the inspector opens that passage.

PART 7

The agent: when a fixed pipeline is not enough

The four modes always do the same steps. Some questions need decisions: no search at all, a question back to the user, a second search based on what the first one found, a calculation, a check of the answer.

IN ONE SENTENCE

The agent is a **LangGraph state machine**: each box is a step (node) that reads and updates a shared state, and the LLM decides which arrow to follow, within hard limits on rounds, rewrites and total steps.

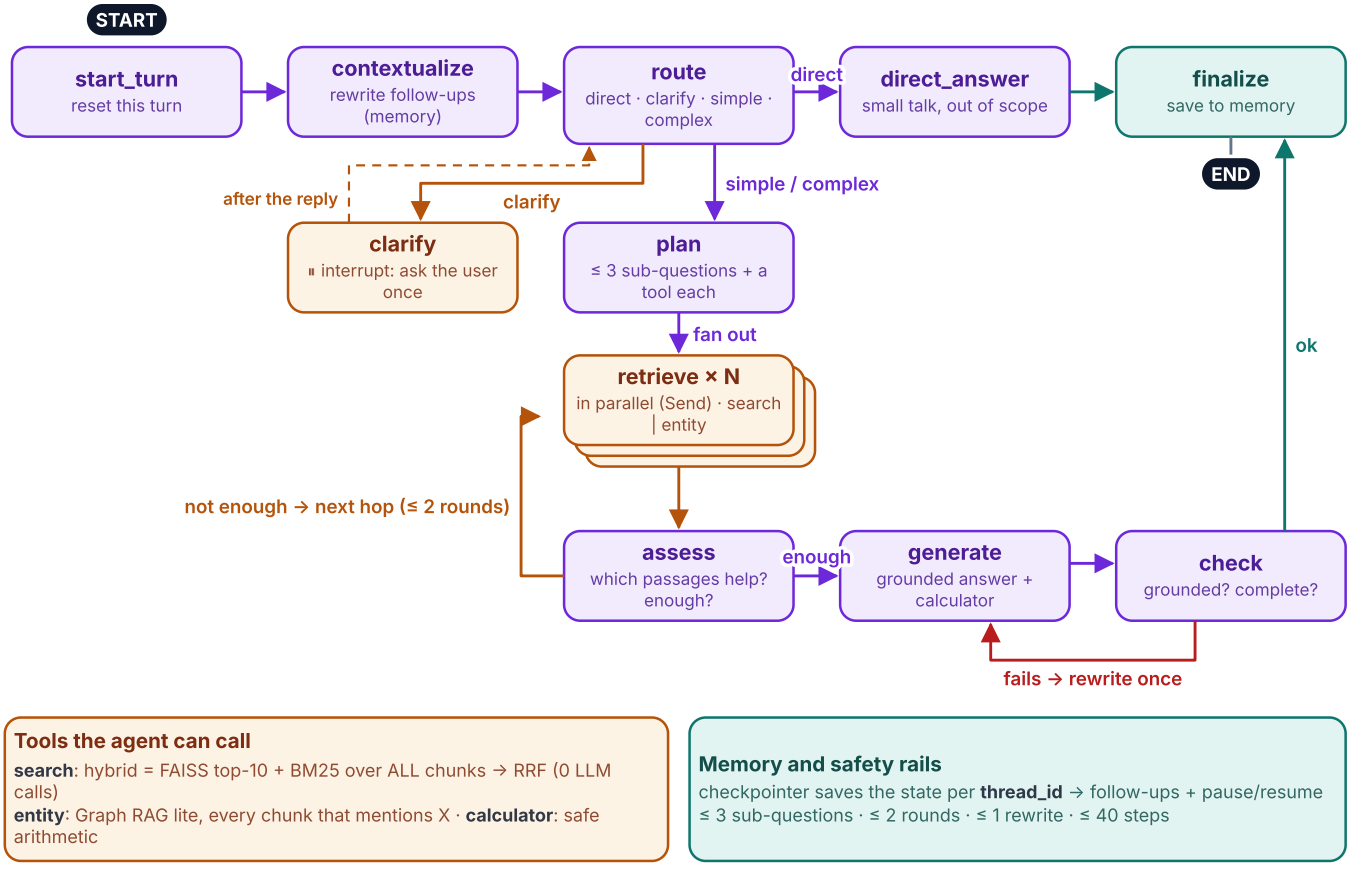


Figure 10 · The agent as a LangGraph state machine (app/agent/graph.py). Boxes are nodes, arrows are edges; labelled arrows are decisions.

What each node does

Node	Decides or does	LLM calls
start_turn	Resets the fields of the previous question (the conversation itself is kept)	0
contextualize	Rewrites a follow-up (“and for interns?”) into a full question using the chat history	0–1
route	**direct** (hello, out of scope) · **clarify** (too vague) · **simple** (one fact) · **complex** (several parts, a calculation, “all …”)	1
clarify	Pauses the graph with `interrupt()` and asks the user once; the reply resumes the graph	0

Node	Decides or does	LLM calls
plan	Simple question: one search with the question itself, no LLM call. Complex: ≤ 3 sub-questions, each with a tool (search or entity)	0–1
retrieve ×N	Runs every sub-question in parallel (LangGraph `Send`); results are merged by reducers	0
assess	Grades the passages: which are relevant, is it enough? If not, writes up to 2 follow-up searches (multi-hop)	1 per round
generate	Grounded answer; any arithmetic is written as `<calc>1.5 * 7</calc>` and computed exactly by a safe calculator	1
check	Is every claim supported, every part answered? If not, one rewrite with feedback	1
finalize	Saves the answer to the conversation memory	0

Typical cost: a direct reply takes 1 LLM call, a simple question 4–5, a complex one 5–7 (MQR + RRF: always 2). The **checkpointer** stores the whole state after every step under a `thread_id`; that is what makes follow-up questions and pause/resume after a clarification possible.

FILES TO OPEN

`app/agent/graph.py` (the wiring) · `nodes.py` (every node) · `state.py` (the shared state and reducers) · `tools.py` · `prompts.py` · `service.py` (run, resume, stream)

TRY IT

Agentic RAG tab: run the 8 scenario buttons. Watch the node-by-node trace on the right, then press **Compare with MQR + RRF** under an answer to see what the fixed pipeline does with the same question.

PART 8

The backend: one request, start to finish

FastAPI turns HTTP requests into Python calls. Knowing the endpoints and following one request end to end is the fastest way to understand how the pieces connect.

IN ONE SENTENCE

The browser sends JSON to an endpoint in `app/routes.py`, the endpoint calls `RagService` or `AgentService`, and everything needed to explain the answer (sources, ranks, timings, call counts) comes back in the reply.

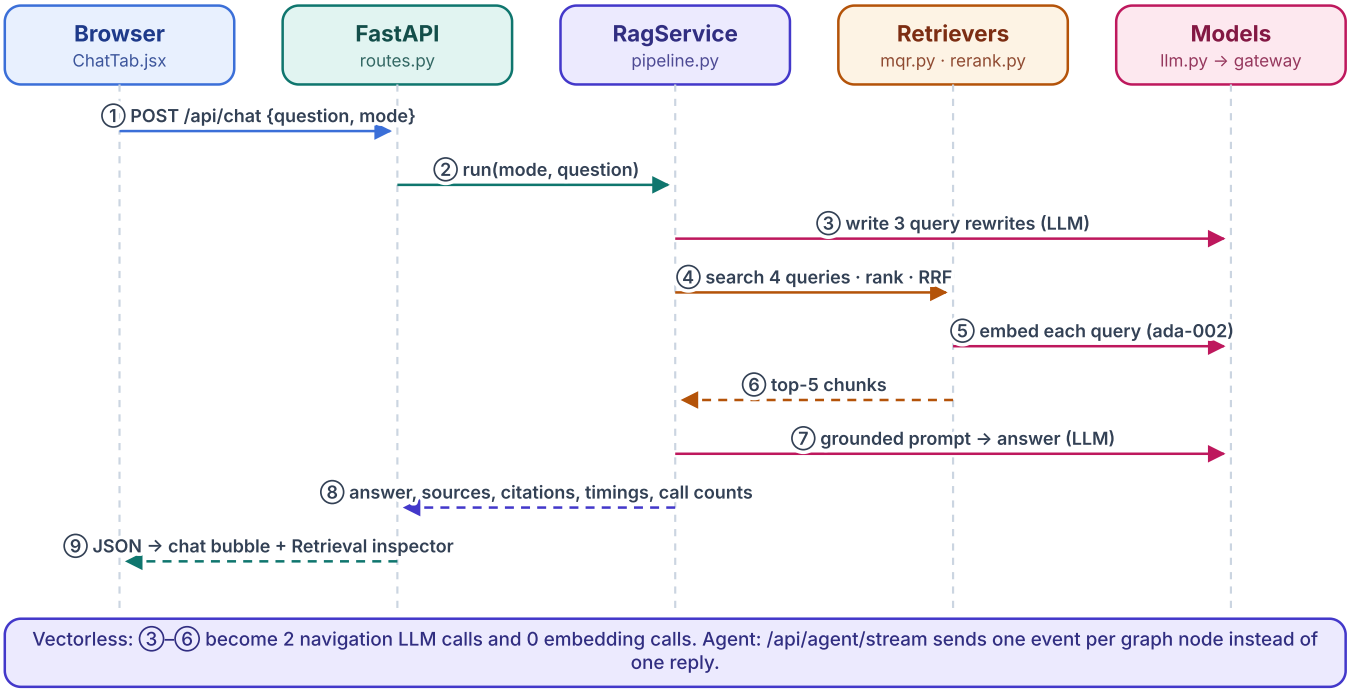


Figure 11 · One chat request in MQR + RRF mode, start to finish. Solid arrows are calls, dashed arrows are replies.

What happens at startup

`app/main.py` creates the `RagService`, which builds the tree (free) and then the FAISS index (one embedding request for 59 chunks). If the embedding model fails, the server still starts and shows the error in a banner; vectorless mode keeps working because it needs no embeddings. Then it creates the `AgentService` (entity graph + compiled LangGraph app) and serves the pre-built React app from `frontend/dist` at `/`.

All endpoints

Method and path	Purpose	Used by
GET /api/health	Models in use, chunk count, tree status, current settings	top bar
GET /api/documents · /api/documents/{name}/chunks	List the handbook files; show how one file was chunked	Knowledge base
POST /api/documents · /api/reindex	Upload a .md/.txt file (≤ 1 MB) and rebuild all indexes	Knowledge base
POST /api/chat	`{question, mode}` → answer + full retrieval details	Lab 1 chat

Method and path	Purpose	Used by
POST /api/compare	The same question in all four modes	Compare
POST /api/evaluate	Retrieval quality and cost per mode on the 15-question set	Evaluate
GET /api/tree · POST /api/tree/summarize	The tree with signposts; (re)write signposts in llm, extractive or none mode	Tree index
POST /api/agent/chat · /resume · /stream	Run the agent; resume after a clarification; stream one event per node	Agentic RAG
GET /api/agent/graph · /scenarios	The graph as a Mermaid diagram; the 8 demo questions	Agentic RAG
GET /api/graph/entities · /api/graph/entity/{name}	Graph RAG lite lookups	Agentic RAG

TRY IT

Open `http://127.0.0.1:8000/docs` : FastAPI's Swagger page lets you call every endpoint by hand and see the exact JSON each screen receives.

PART 9

The frontend: what each tab shows

The React app is the window into the system. Each tab answers one question a learner or a trainer has about it.

IN ONE SENTENCE

Six tabs, built with React and Vite in `frontend/src/`; `api.js` holds every HTTP call, and the pre-built copy in `frontend/dist/` is what FastAPI serves.

Tab	Question it answers	What to look at
Agentic RAG	How does an agent decide what to do?	Route chip (direct, clarify, simple, complex), the live node trace, the mini graph map, the clarification pause, follow-ups in the same thread
Lab 1 chat	How did this mode find these passages?	Mode switch (4 modes) and the Retrieval inspector: queries, the chunk pool with semantic / BM25 / RRF ranks, passages sent, cost and timings; for Vectorless, the navigation path and the nodes the LLM saw
Compare	Same question, which mode does best?	Four columns: chunks found and sent, context tokens, latency, LLM and embedding calls, the answer and its passages
Evaluate	Which mode is better on average, and at what cost?	Hit rate, MRR, results per question type, LLM and embedding calls per question, time, context size; a ✓/× table per question
Tree index	What does the vectorless navigator actually read?	The outline with signposts and their source (llm, cache, extractive); switch signpost modes; the full text of any section
Knowledge base	What is in the index?	Files and chunk counts, every chunk of a file, upload, re-index

How the screens talk to the API

Every tab calls functions in `frontend/src/api.js`, which wraps `fetch()` and turns an error reply into an error message with the backend's hint. The agent tab is the exception: it reads a **Server-Sent Events** stream from `/api/agent/stream`, so each graph step appears as soon as it finishes.

FILES TO OPEN

`frontend/src/App.jsx` (tabs) · `api.js` · `Inspector.jsx` · `AgentTab.jsx` · `TreeTab.jsx` · `EvaluateTab.jsx`

TRY IT

Change the UI: `cd frontend && npm install && npm run dev` starts a live-reloading copy on port 5173 that talks to the API on port 8000. `npm run build` updates `frontend/dist`.

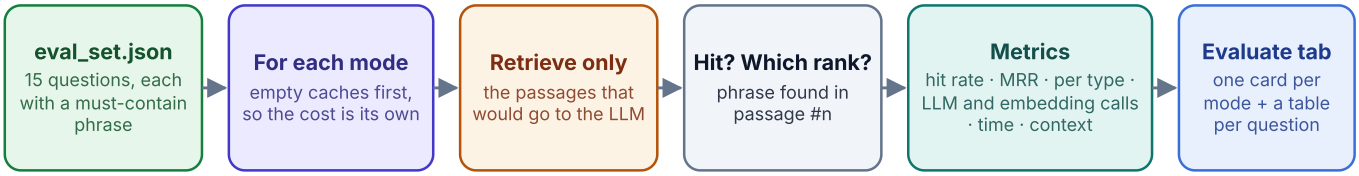
PART 10

Measuring: evaluation and comparison

Opinions about retrieval are cheap. The Evaluate tab turns “I think MQR + RRF is better” into numbers, for quality *and* for cost.

IN ONE SENTENCE

15 questions, each with a phrase the right passage must contain, are run through every mode with empty caches; a question is a **hit** when the phrase is in any retrieved passage, and each mode's own LLM calls, embedding calls and time are counted.



Hit rate = share of questions where the phrase is in any retrieved passage. **MRR** = average of 1 / rank of the first such passage (1.0 = always first).

Figure 12 · The Evaluate tab (app/pipeline.py: evaluate). Retrieval only: no answers are generated.

The 15 questions

Type	How many	Example	Weakness it targets
exact-id	4	PAY-E4035 keeps coming in prod, any idea?	look-alike codes for embeddings
paraphrase	6	Can I WFH on Fridays?	the user's words are not the handbook's words
vague	3	deploy on 30th?	too little text to embed well
multi-part	2	As a grad trainee, how much learning budget do I get and do I carry the on-call pager?	the parts live in different documents

How to read the results

Look per type, not only the average

A mode can win paraphrase questions and lose exact-id ones. The per-type rows show where each mode is strong.

Quality always comes with a cost

Compare hit rate with LLM calls per question, embedding calls, time and context size. A small gain for double the calls may not be worth it.

Cold caches, fair numbers

Each mode starts with empty caches, so MQR + RRF pays for its own rewrites. One run costs about 60 chat and 135 embedding calls.

FILES TO OPEN

`data/eval_set.json` · `app/pipeline.py` (`evaluate()`) · `frontend/src/EvaluateTab.jsx`

TRY IT

Run the evaluation, then switch the tree signposts to *Titles only* on the Tree index tab and run it again. How much does the vectorless hit rate depend on the signposts?

PART 11

Configuration, tests and project map

The behaviour of the app is controlled by `.env`, protected by 64 offline tests, and shipped as two zips built from one solution tree.

IN ONE SENTENCE

Change a number in `.env` to change how much the app searches and sends; run `pytest` to check everything without calling any model; build the student starter with `tools/make_variants.py`.

The most useful .env settings

Setting	Default	What it changes
LLM_MODE	enterprise	Gateway with Azure AD token, or direct API key (Part 4)
CHUNK_SIZE · CHUNK_OVERLAP	500 · 60	Size of the chunks in the vector index (re-index after changing)
BASIC_K	5	Chunks Basic RAG sends
MQR_NUM_QUERIES · MQR_K · MQR_INCLUDE_ORIGINAL	3 · 5 · true	Rewrites, chunks per query, also search the original question
RRF_K · FINAL_K	60 · 5	The constant in 1 / (k + rank); chunks kept after fusion
MAX_CONTEXT_CHARS	6000	Context budget of the grounded prompt
VECTORLESS_MAX_DOCS · _SECTIONS · _STEPS	3 · 4 · 3	How many documents and sections the navigator may pick; navigation calls per question
VECTORLESS_SUMMARIES · VECTORLESS_CACHE	llm · data/.cache/...	Signposts: llm, extractive or none; where LLM signposts are cached
AGENT_MAX_SUBQUESTIONS · _ROUNDS · _REGENERATIONS	3 · 2 · 1	The agent's safety rails
AGENT_SEARCH_MODE · AGENT_RECURSION_LIMIT	hybrid · 40	The agent's search tool (hybrid or mqr_rrf); hard cap on graph steps

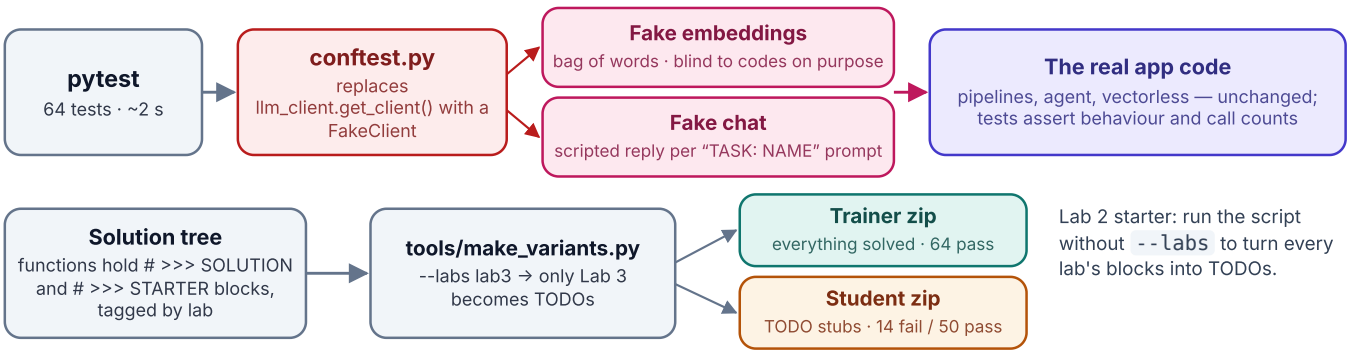


Figure 13 · How the tests run offline and free, and how one solution tree becomes a trainer zip and a student zip.

Project map

```
llm_client.py            how to reach the models (enterprise | direct)
00_check_setup.py        run first: token and one chat call
app/
  main.py                starts the app: indexes, agent, serves the UI
  config.py              RAG and vectorless settings from .env
  llm.py                 chat_complete(), GatewayEmbeddings, RagError, call counters
  ingest.py              files → chunks                        (Lab 1)
  vector_store.py        FAISS index in memory                  (Lab 1)
  basic_rag.py           mode 1: Basic RAG                      (Lab 1)
  mqr.py                 mode 2: Multi-Query Retriever          (Lab 1)
  rerank.py              mode 3: BM25, semantic rank, RRF       (Lab 1)
  generation.py          grounded prompt, budget, citations     (Lab 1)
  pipeline.py            RagService: runs a mode, compare, evaluate
  routes.py · schemas.py the HTTP API and its request shapes
  agent/                 Lab 2: graph.py, nodes.py, state.py, tools.py, prompts.py,
                         graph_rag.py, service.py, routes.py
  vectorless/            Lab 3: tree.py, navigator.py, prompts.py
frontend/src/            React screens; frontend/dist/ = the built copy
data/docs/               the handbook (12 .md files)
data/eval_set.json       15 evaluation questions
data/agent_scenarios.json 8 agent demo questions
tests/                   64 offline tests (fake models)
tools/make_variants.py   trainer zip → student starter
LAB3_VECTORLESS.md       the Lab 3 exercise sheet
```

PART 12

Putting it together

One question, followed through every mode, ties the whole guide together.

IN ONE SENTENCE

There is no best retriever, only a best retriever *for a kind of question and a budget*: the same question can take milliseconds and one passage, or several LLM decisions and a readable path. That is why the app keeps every mode side by side, and why Evaluate reports cost next to quality.

Following “PAY-E4035 keeps coming in prod, any idea?”

Mode	What happens (Parts 3–7)	What you see in the UI
Basic	The question is embedded (1 call) and FAISS returns the 5 nearest chunks. Error-code sections look alike as vectors, so a sibling code such as PAY-E4033 can rank close to PAY-E4035.	5 passages, no citations; check whether 04.4's text is among them
MQR	The LLM rewrites the question 3 times, keeping the code unchanged; 4 FAISS searches; the union of everything found is sent.	4 queries in the inspector, a pool of 10+ chunks, all sent
MQR + RRF	The pool is ranked by meaning and by BM25. BM25 matches the token `pay-e4035` exactly, so that chunk rises; RRF keeps the top 5.	Sem. and BM25 rank columns; the PAY-E4035 chunk near the top; cited passages highlighted
Vectorless	Level 1: the LLM reads the 12 document signposts and opens 04 (payment error codes). Level 2: it picks 04.4, whose title contains the code. The whole section is sent.	Navigation path 04 → 04.4 with the LLM's reasoning; 0 embedding calls
Agent	Typically routed *simple*: one search with the question itself (no planning call), hybrid search = FAISS + BM25 over all chunks, assess says enough, then generate, check, finalize.	Node trace: route → plan → retrieve → assess → generate → check

Which mode when?

Situation	Good choice	Why
Very high traffic, answers in under a second	Basic or MQR + RRF	One or two LLM calls, millisecond retrieval
Users write casually; documents are formal	MQR + RRF	Rewrites bridge the vocabulary gap; RRF keeps the context small
Exact identifiers: error codes, policy ids, SKUs	MQR + RRF, Vectorless, or the agent's hybrid search	All use keyword matching or titles that contain the code
Long, well-structured documents; audits need to know why	Vectorless	Follows the document's own structure; the path is the explanation
Questions with several parts, follow-ups, calculations, or too vague to search	Agent	Plans, loops, calculates, remembers and can ask back
Thousands of short, unstructured texts (tickets, chats)	Vector + BM25 (MQR + RRF style)	No table of contents to navigate

APPENDIX

Glossary

Term	Meaning
Agent	An LLM that decides the next step inside a loop of steps, instead of following a fixed pipeline.
BM25	Keyword relevance formula: rare words of the query that appear in a text raise its score.
Checkpointer	LangGraph component that saves the agent's state after every step, per thread_id.
Chunk	A small piece of a document (here ≤ 500 characters) that is indexed and retrieved on its own.
Citation	A [n] in the answer that points to passage n of the context.
Cosine similarity	How close two vectors point: 1.0 = same direction (same meaning), 0 = unrelated.
Embedding	A list of numbers (1,536 for ada-002) that represents the meaning of a text.
Entity graph	Graph RAG lite: named things (codes, roles, systems) linked when they appear in the same chunk.
FAISS	Meta's library for fast vector search; here an exact in-memory index.
FIL AI Gateway	FIL's proxy to the models; requires an Azure AD token; routes by model name.
Grounded prompt	A prompt that only allows answers from the given passages, with citations.
Hit rate	Share of evaluation questions where the right passage was retrieved.
Interrupt	LangGraph pause: the agent asks the user a question and resumes with the reply.
LangGraph	Library for building agents as graphs of nodes with shared state and conditional edges.
LLM	Large language model; here azure/gpt-6-luna.
MQR	Multi-Query Retriever: the LLM rewrites the question so several searches can be run.
MRR	Mean reciprocal rank: the average of 1 / (position of the first correct passage).
Multi-hop	Answering needs a second search based on what the first one found.
RAG	Retrieval-Augmented Generation: retrieve passages, add them to the prompt, generate.
RagError	The app's error type: an HTTP status plus a hint for the user.
RRF	Reciprocal Rank Fusion: merge rankings by adding 1 / (k + rank) from each list.
Signpost	The one-sentence summary of a tree node that the vectorless navigator reads.
Tree index	The handbook's outline (documents → sections) used by vectorless RAG.
Vectorless RAG	Retrieval by LLM reasoning over the document structure, with no embeddings or vector database.